Bistrô Connect: guia técnico para quem vai mexer no código

Plataforma de cardápio online e pedidos para várias lojas (padaria, café, hamburgueria, pizzaria, doceria, restaurante). Este documento explica como o sistema é organizado, onde mexer e como publicar. Leia também `SEGURANCA.md` e `AUDITORIA.md`.

1. Visão geral

- **Site estático** (HTML, CSS e JavaScript puro, sem framework e sem etapa de build no navegador) hospedado na **Hostinger** (`public_html` , Apache, PHP 8.3).
- **Banco e regras** no **Supabase** (PostgreSQL): tabelas, segurança por linha (RLS) e funções SQL (RPC) para tudo que não pode ser decidido no navegador.
- **Funções de servidor** no Supabase (Edge Functions, TypeScript/Deno): somente a integração do WhatsApp.
- **Ferramentas de apoio** em Python e Node (geram painel, PDFs, exemplos e testes). Não vão para o ar.

```
Cliente (celular)
  loja.html + assets/loja.js --RPC create_order--> Supabase (Postgres)
                                                          |
Equipe (painel)                                           | tempo real
  painel/index.html + painel.js <-----------------------+

Cliente (rastreio)
  /p/<codigo> -> pedido.php + pedido.html --RPC cartao_pedido--> Supabase

WhatsApp (Meta)
  Edge Function whats-webhook -> tabelas whats_* -> aba Atendimento
```

2. Linguagens e peças

Peça	Linguagem	Onde
Páginas e painéis	HTML, CSS, JavaScript	`plataforma/`
Banco, segurança e regras de negócio	SQL (PostgreSQL, PL/pgSQL)	`plataforma/supabase/*.sql`
WhatsApp (receber e enviar)	TypeScript (Deno)	`plataforma/supabase/functions/`
Prévia do link de rastreio	PHP	`plataforma/pedido.php` , `pedido-lib.php`
Links curtos e proteção de arquivos	Apache (`.htaccess`)	`plataforma/.htaccess`
Geradores (painel, CRM, PDFs, CSP)	Python	`ferramentas/`
Testes e capturas de tela	Node.js + Playwright	fora do repositório (ver seção 9)

Biblioteca de terceiros no navegador: `@supabase/supabase-js` (arquivo local em `assets/vendor/`) e `qrcode-generator` (MIT).

3. Mapa de pastas

```
plataforma/                 o que vai para o public_html
  index.html                pagina de vendas
  entrar.html               cadastro, login e criacao da loja
  loja.html, assets/loja.*  cardapio publico (/l/<slug>)
  pedido.html, pedido.php   rastreio (/p/<codigo>) e previa no WhatsApp
  painel/                   painel da loja (GERADO, ver secao 6)
  admin/                    painel do administrador
  crm/                      CRM de leads (GERADO)
  assets/config.js          configuracao publica (URL e chave anon)
  supabase/                 SQL e Edge Functions (bloqueados no Apache)
paineis/loja/               FONTE do painel da loja
ferramentas/                geradores Python (bloqueados no Apache)
manual/                     PDFs de venda e manuais
sites/, modelos/            sites completos de exemplo
```

4. Banco de dados (Supabase)

Tabelas principais: `lojas` , `membros` , `convites` , `categories` , `products` , `orders` , `order_events` , `planos` , `plataforma_admins` , e do WhatsApp `whats_contas` , `whats_segredos` , `whats_conversas` , `whats_mensagens` .

Regra de ouro: o navegador só usa a chave `anon`. Tudo que envolve preço, cupom, permissão ou dados de outras pessoas passa por função SQL `security definer` com `set search_path = ''`, ou por uma política RLS. Nunca coloque a `service_role` em arquivo do site.

Funções mais importantes:

- `create_order(slug, idem, pedido)`: cria o pedido do cardápio. **Recalcula** preços, opcionais, taxa, cupom e mínimo no servidor; é idempotente (`idem`).
- `criar_pedido_painel(loja, idem, pedido)`: pedido criado pela equipe; reutiliza `create_order` (ignora loja pausada e limite de pedidos seguidos).
- `rastrear_pedido(token)` e `cartao_pedido(código)`: dados públicos do rastreio e do cartão; só com o código secreto do pedido.
- `pode(loja, papéis)` e `papel_na_loja(loja)`: base das políticas. Dono e gerente exigem login em 2 etapas (`aal2`).
- `confere_cupom`, `faturamento_loja`, `convidar_membro`, `aceitar_convite`, `criar_loja`, `admin_lojas`.
- WhatsApp: `whats_recebe`, `whats_saida`, `whats_status` (usadas só pelas Edge Functions, com a chave de serviço) e `whats_responder`, `whats_modo`, `whats_robo`, `whats_avisa`, `whats_midia` (usadas pelo painel).

Papéis na loja: `dono`, `gerente`, `atendente`, `cozinha`. A cozinha só vê pedidos e não mexe em pagamento (gatilho `pedido_alterado`).

Ordem dos arquivos SQL

`01-plataforma.sql` é a instalação **completa e atual**: já contém o que está nos arquivos 02, 04, 05 e 07 a 12. O `03-crm.sql` (tabela de leads do CRM) é **separado**: em banco novo, rode o `01` e depois o `03`. Os demais arquivos são **correções incrementais** para bancos que já existem e podem ser rodados mais de uma vez (idempotentes). Em banco existente que parou no `06`, rode `07, 08, 09, 10, 11, 12` nesta ordem. Depois de rodar SQL novo: `notify pgrst, 'reload schema';`.

---

5. Funções de servidor (Edge Functions)

Pasta `plataforma/supabase/functions/` (guia completo em `LEIA-ME-WHATSAPP.md`).

- `whats-webhook`: recebe da Meta, confere assinatura (`X-Hub-Signature-256`), grava a conversa e o robô responde (cardápio, status do pedido, horário, entrega, pagamento, atendente).
- `whats-enviar`: envia respostas da equipe (login do usuário) e abre mídia do cliente.
- Publicação: cole o `index.ts` no editor do Supabase (Edge Functions) e **desligue "Verify JWT"** nas duas.
- Segredos (Supabase > Edge Functions > Secrets): `WA_VERIFY_TOKEN`, `WA_APP_SECRET`, `WA_TOKEN`. O Supabase já fornece `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`.

- Celular do Brasil que chega sem o 9 é enviado com o 9 (`comNove`); o mesmo celular com ou sem 9 cai na mesma conversa (`whats_chave_fone`).

6. Arquivos gerados: NÃO edite à mão

`plataforma/painel/` e `plataforma/crm/` são **gerados**. Se editar direto, a próxima geração apaga a mudança.

- Fonte do painel: `paineis/loja/*` + os trechos `ferramentas/painel-*.pyfrag` (opcionais, seções, cupom, atendimento, pedido manual, visual).
- Gerar: `python3 ferramentas/montar-painel-plataforma.py` (e `python3 ferramentas/montar-crm-plataforma.py` para o CRM).
- Em seguida **sempre** rode `python3 ferramentas/csp.py` (grava a Content-Security-Policy; sem isso, scripts inline editados deixam de funcionar).
- A constante `VERSAO` no gerador troca o `?v=` dos arquivos para furar o cache do navegador. Aumente a cada publicação.
- Trechos `.pyfrag` são código Python executado pelo gerador; usam a função `troca(texto, velho, novo)` , que **falha** se o trecho não for encontrado exatamente uma vez. Isso é proposital: avisa quando a base mudou.

7. Publicar

1. **Banco:** rode os SQL novos no Supabase (SQL Editor).
2. **Funções:** cole os `index.ts` alterados no editor das Edge Functions e publique.
3. **Site:** envie o conteúdo de `plataforma/` para o `public_html` , **sem sobrescrever** `assets/config.js` **de produção** (ele tem a URL e a chave reais). O repositório guarda um `config.js` vazio de propósito.
4. `.htaccess` já bloqueia `ferramentas/` , `supabase/` , `historico/` , `paineis/` e extensões como `.sql` , `.py` , `.md` no site publicado.

8. Segurança (resumo)

- Só a chave `anon` no navegador; `service_role` somente nos segredos das Edge Functions.
- RLS ligada e forçada em todas as tabelas; políticas por papel; funções `security definer` fixam `search_path` .
- Login em 2 etapas (TOTP) para dono e gerente, exigido no banco (`aal2`), não só na tela.
- Rastreio e cartão só com código secreto (16 hex ou 64 caracteres); sem dados pessoais na URL.
- Content-Security-Policy por página; textos do usuário sempre escapados antes de entrar no HTML.

- Token do WhatsApp nunca no navegador. `whats_segredos` não tem permissão para `anon` nem `authenticated`.

---

9. Testes

Os testes automáticos que usei no desenvolvimento (Node + Playwright) e o ambiente local (PostgreSQL 16, PostgREST e Apache simulando o Supabase e a Hostinger) ficaram **fora do repositório**, numa pasta temporária de trabalho. Para um novo desenvolvedor, o caminho mais curto é:

1. Criar um projeto Supabase de teste, rodar o `01-plataforma.sql`.
2. Preencher `plataforma/assets/config.js` local com a URL e a chave `anon` do projeto de teste.
3. Servir `plataforma/` com qualquer servidor estático com PHP (ou o Apache) e testar o fluxo: criar loja, cadastrar produto, fazer pedido, acompanhar no painel. Um bom próximo passo é versionar uma suíte de testes no repositório.

---

10. Pendências conhecidas

- **WhatsApp no painel:** pronto no código, mas o envio fica travado pela Meta enquanto a conta for a de **teste**: token de 24h e restrição de envio para o Brasil (erro 130497). Para ligar de verdade: empresa com CNPJ, verificação da empresa na Meta, número real da loja e token permanente de usuário do sistema.
- Preço do **site personalizado** e do serviço de **montagem do cardápio**: definidos sob orçamento, sem valor fixo.
- Painel mostra "não enviada" sem o motivo; vale guardar o código de erro da Meta (`statuses[].errors`).
- Falta uma suíte de testes versionada e um processo de publicação automatizado (hoje é manual, por envio de arquivos).
- Termos de uso: faltam razão social, CNPJ e cidade em `assets/config.js` (`empresa`).